

EXPERIMENT - 5

Title

SQL Views, View Updatability and Recursive CTE

Aim

To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display employee reporting chains.

Objectives

1. To understand the concept of SQL views.
2. To create a view for department salary summary.
3. To create a view for employee hierarchy.
4. To test the updatability of SQL views.
5. To understand recursive Common Table Expressions.
6. To display employee reporting chains using a recursive CTE.

Software/ Tools Required

- ByteXL / Nimbus
- MySQL
- SQL

Theory

A **view** is a virtual table created from the result of an SQL query. It does not normally store the data separately. Instead, it displays data from one or more underlying tables.

Views are useful for simplifying complex queries, improving data security, and presenting only required information to users.

1. Department Salary Summary View

A department salary summary view can be created using aggregate functions such as COUNT(), SUM(), and AVG(). It can display the number of employees, total salary, and average salary for each department.

2. Employee Hierarchy View

An employee hierarchy represents the relationship between employees and their managers. A self-referencing ManagerID column can be used to identify the manager of each employee.

3. View Updatability

Some SQL views are updatable, while others are not.

A simple view based on one table and without aggregate functions or grouping can generally be updated. Views containing GROUP BY, aggregate functions, joins, or other complex operations are generally not directly updatable.

4. Recursive CTE

A **Recursive Common Table Expression (CTE)** is used to process hierarchical or tree-like data. It consists of two parts:

- Anchor query - provides the starting records.
- Recursive query - repeatedly finds related records.

A recursive CTE can be used to display an employee's reporting chain from the top-level manager to lower-level employees.

SQL Implementation

1. Add ManagerID to Employee Table

The ManagerID column is required to represent the employee-manager relationship.

```
ALTER TABLE Employee
ADD ManagerID INT NULL;
```

2. Add Self-Referencing Foreign Key

```
ALTER TABLE Employee
ADD CONSTRAINT fk_manager
FOREIGN KEY (ManagerID) REFERENCES Employee(EmpID);
```



fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	30	0	Records: 30 Duplicates: 0 Warnings: 0	2	0	0

3. Assign Managers

```
UPDATE Employee
SET ManagerID = CASE
    WHEN EmpID IN (1, 6, 11, 16, 21) THEN NULL
    WHEN EmpID BETWEEN 2 AND 5 THEN 1
    WHEN EmpID BETWEEN 7 AND 10 THEN 6
    WHEN EmpID BETWEEN 12 AND 15 THEN 11
    WHEN EmpID BETWEEN 17 AND 20 THEN 16
    WHEN EmpID BETWEEN 22 AND 25 THEN 21
    WHEN EmpID = 26 THEN 1
    WHEN EmpID = 27 THEN 6
    WHEN EmpID = 28 THEN 11
```

```

        WHEN EmpID = 29 THEN 16
        WHEN EmpID = 30 THEN 21
    END;

```

MySQL - db_454rqctf2

Query 1

```

216 UPDATE Employee
217 SET ManagerID = CASE
218     WHEN EmpID IN (1, 6, 11, 16, 29) THEN NULL
219     WHEN EmpID BETWEEN 2 MO 5 THEN 1
220     WHEN EmpID BETWEEN 7 MO 10 THEN 6
221     WHEN EmpID BETWEEN 12 MO 15 THEN 11
222     WHEN EmpID BETWEEN 17 MO 20 THEN 16
223     WHEN EmpID BETWEEN 22 MO 25 THEN 21
224     WHEN EmpID = 27 THEN 6
225     WHEN EmpID = 28 THEN 11
226     WHEN EmpID = 29 THEN 16
227     WHEN EmpID = 30 THEN 21
228 END;
229

```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
JO			Rows matched: 10 Changed: 25 Warnings: 0	34		25

4. Display Employee Manager Relationship

```

SELECT EmpID, EmpName, ManagerID
FROM Employee
ORDER BY EmpID;

```

MySQL - db_454rqctf2

Query 1

```

231 SELECT EmpID, EmpName, ManagerID
232 FROM Employee
233 ORDER BY EmpID;

```

EmpID	EmpName	ManagerID
1	Aarav Sharma	null
2	Priya Singh	
3	Rohan Verma	
4	Ananya Gupta	
5	Kunal Mehta	
6	Neha Kapoor	null
7	Rahul Vaidya	
8	Sneha Jain	
9	Amit Kumar	
10	Ishita Roy	
11	Vikas Gupta	null
12	Simran Kaur	11
13	Aditya Mishra	11
14	Pooja Agarwal	11

MySQL - db_454rqctf2			
Query 1			
Query 2			
<pre> 231 SELECT EmpID, EmpName, ManagerID 232 FROM Employee 233 ORDER BY EmpID; </pre>			
15		INDIRA THAKUR	11
16		Nisha Sharma	null
17		Arjun Singh	16
18		Riya Malhotra	16
19		Varun Saxena	16
20		Tanya Bansal	16
21		Dev Patel	null
22		Meera Joshi	21
23		Sahil Khan	21
24		Kavya Nair	21
25		Manish Jain	21
26		Isha Verma	1
27		Yash Thakur	6
28		Aditi Sinha	11
29		Harsh Vardhan	16
30		Diva Arora	31

5. Create Department Salary Summary View

This view displays the number of employees, total salary, and average salary for each department.

```

CREATE VIEW DepartmentSalarySummary AS
SELECT DeptID,
       COUNT(*) AS TotalEmployees,
       SUM(Salary) AS TotalSalary,
       AVG(Salary) AS AverageSalary
FROM Employee
GROUP BY DeptID;

```

MySQL - db_454rqctf2

EXPLORER

Search...

TABLES

department

departments

employee

employees

leaves

project

salaries

FUNCTIONS

GetAverageSalaryOfDepartment

Query 1

Query 2

```
235 CREATE VIEW DepartmentSalarySummary AS
236 SELECT DeptID,
237        COUNT(*) AS TotalEmployees,
238        SUM(Salary) AS TotalSalary,
239        AVG(Salary) AS AverageSalary
240 FROM Employee
241 GROUP BY DeptID;
```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

6. Display Department Salary Summary

```
SELECT* FROM DepartmentSalarySummary;
```

MySQL - db_454rqctf2

Query 1 Query 2 Query 3 Query 4

1 select * from departmentsalarysummary;

DeptID	TotalEmployees	TotalSalary	AverageSalary
101	6	357000.00	59500.000000
102	6	312000.00	52000.000000
103	6	377000.00	62833.333333
104	6	346000.00	57666.666667
105	6	391000.00	65166.666667

7. Create Employee Hierarchy View

This view displays each employee along with the name of their immediate manager.

```
CREATE VIEW EmployeeHierarchy AS
SELECT E.EmpID,
       E.EmpName,
       E.ManagerID,
       M.EmpName AS ManagerName
FROM Employee E
LEFT JOIN Employee M
ON E.ManagerID = M.EmpID;
```

MySQL - db_454rqctf2

244 CREATE VIEW EmployeeHierarchy AS
245 SELECT E.EmpID,
246 E.EmpName,
247 E.ManagerID,
248 M.EmpName AS ManagerName
249 FROM Employee E
250 LEFT JOIN Employee M
251 ON E.ManagerID = M.EmpID;
252

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

8. Display Employee Hierarchy

```
SELECT*
FROM EmployeeHierarchy
ORDER BY EmpID;
```

Aryan Pratap Singh
25LBSC3266

Table 1

1000

MySQL - db_454rqctf2

Query2

Search

262 UPDATE EmployeeSalaryView

263 SET Salary = 1000

264 WHERE EmpID = 2;

department

deptments

departmentsalarysummary

employee

employeehierarchy

employees

employeesalaryview

leaves

project

salaries

fieldCount

affectedRows

insertId

info

serverStatus

warningStatus

changedRows

Rows matched: 1 Orlanged: 1 Warnings: 0

11. Verify Updated Data

```

SELECT *
FROM EmployeeSalaryView
WHERE EmpID = 2;

```

The simple view can be updated because it is based on a single table and does not contain grouping or aggregate functions.

MySQL - db_454rqctf2

Query3

Search

select * from EmployeeSalaryView;

department

deptments

departmentsalarysummary

employee

employeehierarchy

employees

employeesalaryview

leaves

project

salaries

EmpID

EmpName

Salary

GetAverageSalaryOfDepartment1

ApproveOrRejectLeave

	Aarav Sharma	72000.00
	PriyaSingh	66000.00
	RohanVerm11	58000.00
		61000.00
	Kulilll Mehta.	55000.00
	Ncha Kapoor	48000.00
	Rahul Yadav	52000.00
	SnehaJain	60000.00
	AmitKotTWlr	45000.00
10	Id.Jt., Roy	57000.00
	Vik.ts Gupta	68000.00
12	SImran Katar	62000.00
13	AdityillMishra	59000.00
	PoojaAgaiwal	51000.00

The screenshot shows a MySQL IDE window titled 'MySQL - db_454rqctf2'. On the left, the 'EXPLORER' pane lists database objects, with 'employeesalaryview' selected under the 'VIEWS' section. The main query editor shows 'Query 1' with the statement: `1 select * from employeesalaryview;`. The results pane displays a table with 3 columns and 11 rows of data.

15	Monit Iwari	73000.00
16	Nisha Sharma	56000.00
17	Arjun Singh	49000.00
18	Riya Malhotra	63000.00
19	Varun Saxena	54000.00
20	Tanya Bansal	67000.00
21	Dev Patel	75000.00
22	Meera Joshi	66000.00
23	Sahil Khan	58000.00
24	Kavya Nair	70000.00
25	Manish Jain	53000.00
26	Isha Verma	46000.00
27	Yash Thakur	50000.00
28	Aditi Sinha	64000.00
29	Harsh Vardhan	57000.00

12. Test Department Salary Summary View

The following statement demonstrates that an aggregate view is not directly updatable.

```
UPDATE DepartmentSalarySummary
SET AverageSalary = AverageSalary + 1000
WHERE DeptID = 101;
```

This view is not directly updatable because it contains aggregate functions and GROUP BY.



13. Recursive CTE for Reporting Chain

The following recursive CTE displays employees along with their reporting levels.

```
WITH RECURSIVE EmployeeChain AS (
    SELECT
        EmpID,
        EmpName,
```

```

        ManagerID,
        0 AS Level
    FROM Employee
    WHERE ManagerID IS NULL
UNION ALL
SELECT
    E.EmpID,
    E.EmpName,
    E.ManagerID,
    EC.Level+ 1
    FROM Employee E
    INNER JOIN EmployeeChain EC
    ON E.ManagerID = EC.EmpID
)
SELECT
    EmpID,
    EmpName,
    ManagerID,
    Level
FROM EmployeeChain
ORDER BY Level, EmpID;

```

MySQL - db_454rqctf2

EmpID	EmpName	ManagerID	Level
1	Aarav Sharma	null	0
6	Neha Kapoor	null	0
11	Vikas Gupta	null	0
16	Nisha Sharma	null	0
21	Dev Patel	null	0
2	Priya Singh	1	1
3	Rohan Verma	1	1
4	Ananya Gupta	1	1
5	Kunal Mehta	1	1
7	Rahul Yadav	6	1

MySQL - db_454rqctf2			
EXPLORER	270	WITH RECURSIVE EmployeeChain AS (	
Search...	271	SELECT	
TABLES	272	[EmpID, EmpName, ManagerID,] AS Level	
☐ department	273	FROM Employee	
☐ departments	274	WHERE ManagerID IS NULL	
☐ departmentsalariesummary	275	UNION ALL	
☐ employee	276	SELECT E.EmpID, E.EmpName, E.ManagerID, EC.Level + 1	
☐ employeeehierarchy	8	Sneha Jain	6
☐ employees	9	Amit Kumar	6
☐ employeesalaryview	10	Ishita Roy	6
☐ leaves	12	Simran Kaur	11
☐ project	13	Aditya Mishra	11
☐ salaries	14	Pooja Agarwal	11
FUNCTIONS	15	Mohit Tiwari	11
GetAverageSalaryOfDepartment	17	Arjun Singh	16
PROCEDURES	18	Riya Malhotra	16
ApproveOrRejectLeave	19	Varun Saxena	16
> Saved Queries (17)	20	Tanya Bansal	16
	22	Meera Joshi	21
	23	Sahil Khan	21
	24	Kavya Nair	21
	25	Manish Jain	21
	26	Isha Verma	1
	27	Yash Thakur	6
	28	Aditi Sinha	11
	29	Harish Vardhan	16
	30	Diya Agarwal	21

14. Recursive CTE with Reporting Chain

This query displays the reporting hierarchy in a readable form.

```

WITH RECURSIVE EmployeeChain AS (
    SELECT EmpID, EmpName, ManagerID,
           CAST(EmpName AS CHAR(500)) AS ReportingChain
    FROM Employee
    WHERE ManagerID IS NULL
UNION ALL
SELECT
    E.EmpID, E.EmpName, E.ManagerID,
    CONCAT(EC.ReportingChain, ' -> ', E.EmpName)
    FROM Employee E
    INNER JOIN EmployeeChain EC
    ON E.ManagerID = EC.EmpID
)
SELECT
    EmpID, EmpName, ManagerID, ReportingChain
FROM EmployeeChain
ORDER BY EmpID;

```

MySQL - db_454rqctf2				
EXPLORER	<pre> 285 WITH RECURSIVE EmployeeChain AS (286 SELECT EmpID, EmpName, ManagerID, 287 CAST(EmpName AS CHAR(500)) AS ReportingChain 288 FROM Employee WHERE ManagerID IS NULL 289 UNION ALL 290 SELECT E.EmpID, E.EmpName, E.ManagerID, 291 CONCAT(EC.ReportingChain, ' -> ', E.EmpName) 292 FROM Employee E 293 INNER JOIN EmployeeChain EC 294 ON E.ManagerID = EC.EmpID 295) 296 SELECT EmpID, EmpName, ManagerID, ReportingChain 297 FROM EmployeeChain 298 ORDER BY EmpID; </pre>			
TABLES	EmpID	EmpName	ManagerID	ReportingChain
☐ department	1	Aarav Sharma	null	Aarav Sharma
☐ departments	2	Priya Singh	1	Aarav Sharma -> Priya Singh
☐ departmentsalarysummary	3	Rohan Verma	1	Aarav Sharma -> Rohan Verma
☐ employee	4	Ananya Gupta	1	Aarav Sharma -> Ananya Gupta
☐ employeeehierarchy	5	Kunal Mehta	1	Aarav Sharma -> Kunal Mehta
☐ employees	6	Neha Kapoor	null	Neha Kapoor
☒ employeesalaryview	7	Rahul Yadav	6	Neha Kapoor -> Rahul Yadav
☐ leaves	8	Sneha Jain	6	Neha Kapoor -> Sneha Jain
☐ project	9	Amit Kumar	6	Neha Kapoor -> Amit Kumar
☐ salaries	10	Ishita Roy	6	Neha Kapoor -> Ishita Roy
FUNCTIONS	11	Vikas Gupta	null	Vikas Gupta
☒ GetAverageSalaryOfDepartment				
PROCEDURES				
☒ ApproveOrRejectLeave				

MySQL - db_454rqctf2				
EXPLORER	<pre> 285 WITH RECURSIVE EmployeeChain AS (286 SELECT EmpID, EmpName, ManagerID, 287 CAST(EmpName AS CHAR(500)) AS ReportingChain 288 FROM Employee WHERE ManagerID IS NULL 289 UNION ALL 290 SELECT E.EmpID, E.EmpName, E.ManagerID, 291 CONCAT(EC.ReportingChain, ' -> ', E.EmpName) </pre>			
TABLES	12	Simran Kaur	11	Vikas Gupta -> Simran Kaur
☐ department	13	Aditya Mishra	11	Vikas Gupta -> Aditya Mishra
☐ departments	14	Pooja Agarwal	11	Vikas Gupta -> Pooja Agarwal
☐ departmentsalarysummary	15	Mohit Tiwari	11	Vikas Gupta -> Mohit Tiwari
☐ employee	16	Nisha Sharma	null	Nisha Sharma
☐ employeeehierarchy	17	Arjun Singh	16	Nisha Sharma -> Arjun Singh
☐ employees	18	Riya Malhotra	16	Nisha Sharma -> Riya Malhotra
☒ employeesalaryview	19	Varun Saxena	16	Nisha Sharma -> Varun Saxena
☐ leaves	20	Tanya Bansal	16	Nisha Sharma -> Tanya Bansal
☐ project	21	Dev Patel	null	Dev Patel
☐ salaries	22	Meera Joshi	21	Dev Patel -> Meera Joshi
FUNCTIONS	23	Sahil Khan	21	Dev Patel -> Sahil Khan
☒ GetAverageSalaryOfDepartment	24	Kavya Nair	21	Dev Patel -> Kavya Nair
PROCEDURES	25	Manish Jain	21	Dev Patel -> Manish Jain
☒ ApproveOrRejectLeave	26	Isha Verma	1	Aarav Sharma -> Isha Verma
> Saved Queries (17)	27	Yash Thakur	6	Neha Kapoor -> Yash Thakur
	28	Aditi Sinha	11	Vikas Gupta -> Aditi Sinha
	29	Harsh Vardhan	16	Nisha Sharma -> Harsh Vardhan
> Saved Queries (17)	30	Diya Agarwal	21	Dev Patel -> Diya Agarwal

Result

SQL views for department salary summary and employee hierarchy were successfully created. The updatability of simple and aggregate views was tested. A recursive CTE was successfully implemented to display employee reporting levels and reporting chains.

Learning Outcomes

After completing this experiment, we are able to:

Aryan Pratap Singh
25LBCS3266

1. Create and use SQL views.
2. Create summary views using aggregate functions.
3. Represent employee-manager relationships.
4. Understand which views are updatable.
5. Use recursive CTEs for hierarchical data.
6. Display employee reporting chains.

Conclusion

This experiment provided practical knowledge of SQL views, view updatability, employee hierarchies, and recursive CTEs. The required views were created successfully, their behavior was tested, and employee reporting chains were displayed using recursive SQL queries.